

P2733R1

Fix handling of empty specifiers in `std::format`

Published Proposal, 2023-02-06

Authors:

[Victor Zverovich](#)

[Mark de Wever](#)

Audience:

LEWG

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Table of Contents

1 Introduction

2 Changes from R0

3 Proposal

4 LEWG Poll Results

5 Wording

6 Acknowledgements

References

Informative References

§ 1. Introduction

This paper amends the proposed resolution of [\[LWG3776\]](#) per LEWG feedback and makes the necessary changes to the `set_debug_format` API to enable the proposed resolution and fix tuple formatting. It also makes a drive-by fix to the range formatter adding a missing call to `parse` for the underlying formatter.

§ 2. Changes from R0

- Added a comparison with potential alternative resolutions of the nested range/tuple formatting bug.

§ 3. Proposal

[\[LWG3776\]](#) "Avoid parsing format-spec if it is not present or empty" proposed omitting the call to `formatter::parse` for empty format specifiers (*format-spec* in [\[format.string.general\]](#) of [\[N4917\]](#)).

Consider the following example:

```
struct S {};  
  
template <>  
struct std::formatter<S> {  
    constexpr auto parse(format_parse_context& ctx) { return ctx.begin(); }  
    auto format(S, format_context& ctx) const { return ctx.out(); }  
};  
  
int main() {  
    auto s1 = fmt::format("{} ", S()); // (1) no format-spec  
    auto s2 = fmt::format(":{ } ", S()); // (2) empty format-spec  
}
```

In (1) *format-spec* is not present and in (2) it is present but empty. There is nothing to parse in both of these cases and therefore requiring implementations to call `formatter::parse` doesn't make a lot of sense. It only adds unnecessary overhead for the common case which is what [\[LWG3776\]](#) was proposing to eliminate. Implementation experience in `{fmt}` showed that requiring the call to `parse` has negative impact on formatting of ranges where we had to unnecessarily call this function from

multiple places. The same issue may exist in other contexts such as format string compilation. In the tuple case there aren't even nested format specifiers to call the underlying `parse` on.

Additionally [LWG3776] made a drive-by fix, clarifying that the two cases are equivalent which was not obvious from existing wording. This is arguably even more important than omitting `parse`, particularly because formatting of ranges ([P2286]) doesn't allow distinguishing between the two forms for nested specifiers, e.g.

```
auto s = std::format("{::}", std::vector<S>(2));  
//           ^ empty format-spec for S
```

Having the two cases equivalent is also more intuitive and consistent with all existing standard formatters.

Library Evolution Working Group (LEWG) reviewed [LWG3776] in Kona and approved it with the amendment that implementations are allowed but not required to omit the call to `formatter::parse` for empty *format-spec*.

Barry Revzin pointed out an existing limitation of the formatting ranges design that requires calling `set_debug_format` from the `parse` function. However, as discovered by Mark de Wever while implementing ranges formatting in libc++, the `formatter` specialization for tuples already omits the call to `parse` for the underlying type so we need to fix this anyway. The following example illustrates the fix:

```
auto s = fmt::format!("{}", std::make_tuple(std::make_tuple('a')));
```

Before	After
<code>s == ((a))</code>	<code>s == (('a'))</code>

This paper amends the proposed resolution of [LWG3776] per LEWG feedback and makes the necessary changes to the `set_debug_format` API both to enable the proposed resolution and to fix tuple formatting. It also fixes a specification bug in `range_formatter` that doesn't mention calling `parse` for the underlying formatter. This proposal has been implemented in [FMT] and in a branch of [LIBCXX].

Some potential alternative resolutions for the nested range/tuple formatting bug are:

- Make range-based formatters conditionally debug-enabled specializations (S1).
- Always call the underlying formatter `parse` function (S2).
- Default the output type of debug-enabled specializations in the constructor (S3).

The table below compares alternative solutions with the current proposal (S0):

Type	Format	Before	S1 & S2	S3	S0
<code>char</code>	<code>{}</code>	<code>a</code>	<code>a</code>	<code>a</code>	<code>a</code>
<code>char</code>	<code>{:?:}</code>	<code>'a'</code>	<code>'a'</code>	<code>'a'</code>	<code>'a'</code>
<code>vector<char></code>	<code>{}</code>	<code>['a']</code>	<code>['a']</code>	<code>['a']</code>	<code>['a']</code>
<code>vector<char></code>	<code>{::}</code>	<code>[a]</code>	<code>[a]</code>	<code>['a']</code>	<code>[a]</code>
<code>vector<char></code>	<code>{::c}</code>	<code>[a]</code>	<code>[a]</code>	<code>[a]</code>	<code>[a]</code>
<code>vector<char></code>	<code>{::?:}</code>	<code>['a']</code>	<code>['a']</code>	<code>['a']</code>	<code>['a']</code>
<code>map<char, char></code>	<code>{}</code>	<code>{a: a}</code>	<code>{'a': 'a'}</code>	<code>{'a': 'a'}</code>	<code>{'a': 'a'}</code>
<code>set<char></code>	<code>{}</code>	<code>{'a'}</code>	<code>{'a'}</code>	<code>{'a'}</code>	<code>{'a'}</code>
<code>set<char></code>	<code>{::}</code>	<code>{a}</code>	<code>{a}</code>	<code>{'a'}</code>	<code>{a}</code>
<code>set<char></code>	<code>{::c}</code>	<code>{a}</code>	<code>{a}</code>	<code>{a}</code>	<code>{a}</code>
<code>set<char></code>	<code>{::?:}</code>	<code>{'a'}</code>	<code>{'a'}</code>	<code>{'a'}</code>	<code>{'a'}</code>
<code>tuple<char></code>	<code>{}</code>	<code>('a')</code>	<code>('a')</code>	<code>('a')</code>	<code>('a')</code>
<code>vector<vector<char>></code>	<code>{}</code>	<code>[[a]]</code>	<code>[['a']]</code>	<code>[['a']]</code>	<code>[['a']]</code>
<code>vector<vector<char>></code>	<code>{::}</code>	<code>[['a']]</code>	<code>[['a']]</code>	<code>[['a']]</code>	<code>[['a']]</code>
<code>vector<vector<char>></code>	<code>{:::}</code>	<code>[[a]]</code>	<code>[[a]]</code>	<code>[['a']]</code>	<code>[[a]]</code>
<code>vector<vector<char>></code>	<code>{:::c}</code>	<code>[[a]]</code>	<code>[[a]]</code>	<code>[[a]]</code>	<code>[[a]]</code>
<code>vector<vector<char>></code>	<code>{:::?:}</code>	<code>[['a']]</code>	<code>[['a']]</code>	<code>[['a']]</code>	<code>[['a']]</code>
<code>vector<tuple<char>></code>	<code>{}</code>	<code>[(a)]</code>	<code>[('a')]</code>	<code>[('a')]</code>	<code>[('a')]</code>
<code>tuple<tuple<char>></code>	<code>{}</code>	<code>((a))</code>	<code>(('a'))</code>	<code>(('a'))</code>	<code>(('a'))</code>
<code>tuple<vector<char>></code>	<code>{}</code>	<code>[[a]]</code>	<code>[['a']]</code>	<code>[['a']]</code>	<code>[['a']]</code>

S1 and S2 are inconsistent with the resolution of [\[LWG3776\]](#) earlier approved by LEWG and not proposed here. They are only included for completeness.

S3 is similar to the current proposal (S0) and the difference is that in S3 the default of the element type is changed to the debug format. This means that users have to give explicit specifiers to get the default format, e.g. `"{:s}"` instead of `"{:}"`:

```

auto v = std::vector<char>{'a'};
auto s1 = std::format("{:}", v); // ['a'] in S3, [a] in S0
auto s2 = std::format("{:s}", v); // [a] in both S0 and S3

```

On the other hand combining the debug format with other specifiers such as width is easier in S3:

```
auto v = std::vector<char>{'a'};
auto s1 = std::format("{::4}", v); // ['a' ] in S3, [a  ] in S0
auto s2 = std::format("{::4?}", v); // ['a' ] in both S0 and S3
```

§ 4. LEWG Poll Results

POLL: Relax the requirements table 74 and 75 to make the optimization allowed by the issue resolution of LWG3776 a QoI issue with additional changes to the handle class removed

SF	F	N	A	SA
1	9	2	1	1

Outcome: consensus in favour

POLL: Adopt the amended proposed resolution of LWG3776 "Avoid parsing format-spec if it is not present or empty". Return the issue to LWG for C++23 (to be confirmed by electronic polling)

SF	F	N	A	SA
2	6	1	2	1

Outcome: weak consensus in favour

§ 5. Wording

This wording is relative to [\[N4917\]](#).

Modify 22.14.6.1 [\[formatter.requirements\]](#) as indicated:

-3- Given character type `charT`, output iterator type `Out`, and formatting argument type `T`, in Table 74 and Table 75:

...

`pc.begin()` points to the beginning of the *format-spec* (22.14.2 [format.string]) of the replacement field being formatted in the format string. If *format-spec* is **not present or** empty then either `pc.begin() == pc.end()` or `*pc.begin() == '}'`.

Modify *BasicFormatter* requirements [\[tab:formatter.basic\]](#) as indicated:

Expression	Return type	Requirement
<code>f.format(u, fc)</code>	<code>FC::iterator</code>	Formats <code>u</code> according to the specifiers stored in <code>*this</code> , writes the output to <code>fc.out()</code> , and returns an iterator past the end of the output range. The output shall only depend on <code>u</code> , <code>fc.locale()</code> , <code>fc.arg(n)</code> for any value <code>n</code> of type <code>size_t</code> , and the range <code>[pc.begin(), pc.end())</code> from the last call to <code>f.parse(pc)</code> . When the <i>format-spec</i> (22.14.2 [format.string]) is not present or empty the call to <code>f.parse(pc)</code> may be omitted.

Modify *Formatter* requirements [tab:formatter] as indicated:

Expression	Return type	Requirement
<code>f.format(t, fc)</code>	<code>FC::iterator</code>	Formats <code>t</code> according to the specifiers stored in <code>*this</code> , writes the output to <code>fc.out()</code> , and returns an iterator past the end of the output range. The output shall only depend on <code>t</code> , <code>fc.locale()</code> , <code>fc.arg(n)</code> for any value <code>n</code> of type <code>size_t</code> , and the range <code>[pc.begin(), pc.end())</code> from the last call to <code>f.parse(pc)</code> . When the <i>format-spec</i> (22.14.2 [format.string]) is not present or empty the call to <code>f.parse(pc)</code> may be omitted.

In [format.formatter.spec]:

-2- Let `charT` be either `char` or `wchar_`. Each specialization of `formatter` is either enabled or disabled, as described below. A *debug-enabled* specialization of `formatter` additionally provides a public, constexpr, non-static member function `set_debug_format(bool set)` which modifies the state of the `formatter` to be as if the type of the *std-format-spec* parsed by the last call to `parse` were `? if set is true and none otherwise`. Each header that declares the template `formatter` provides the following enabled specializations:

...

In [format.range.formatter]

```
namespace std {
    template<class T, class charT = char>
        requires same_as<remove_cvref_t<T>, T> && formattable<T, charT>
        class range_formatter {
            ...
            constexpr const formatter<T, charT>& underlying() const { return underl
```

```
constexpr range_formatter();

template<class ParseContext>
constexpr typename ParseContext::iterator
    parse(ParseContext& ctx);
};
}
```

...

```
constexpr void set_brackets(basic_string_view<charT> opening, basic_string_
```

Effects: Equivalent to:

```
opening-bracket_ = opening;
closing-bracket_ = closing;
```

```
constexpr range_formatter();
```

Effects: Calls `underlying_.set_debug_format(true)` if it is a valid expression.

```
template<class ParseContext>
constexpr typename ParseContext::iterator
    parse(ParseContext& ctx);
```

Effects: Parses the format specifier `s` as a *range-format-spec* and stores the parsed specifiers in `*this`. The values of `opening-bracket_`, `closing-bracket_`, and `separator_` are modified if and only if required by the *range-type* or the `n` option, if present. ~~If:~~

- ~~the range-type is neither `s` nor `?s`;~~
- ~~`underlying_.set_debug_format()` is a valid expression, and~~
- ~~there is no *range-underlying-spec*;~~

~~then calls `underlying_.set_debug_format()`.~~

If there is *range-underlying-spec*:

- calls `underlying_.set_debug_format(false)` if it is a valid expression,

- calls `underlying_.parse` to parse *range-underlying-spec*.

In `[format.range.fmtstr]`:

```
namespace std {
    template<range_format K, ranges::input_range R, class charT>
        requires (K == range_format::string || K == range_format::debug_string)
        struct range-default-formatter<K, R, charT> {

            ...

        public:
            constexpr range-default-formatter();

            template<class ParseContext>
                constexpr typename ParseContext::iterator
                parse(ParseContext& ctx);

            ...
        };
    }
}
```

```
constexpr range-default-formatter();
```

Effects: Calls `underlying_.set_debug_format(true)` if it is a valid expression and `K == range_format::debug_string`.

```
template<class ParseContext>
    constexpr typename ParseContext::iterator
    parse(ParseContext& ctx);
```

-2- *Effects:* Equivalent to:

```
auto i = underlying_.parse(ctx);
if constexpr (K == range_format::debug_string) {
    underlying_.set_debug_format(true);
}
return i;
if constexpr (K == range_format::debug_string) {
```

```

    underlying_.set_debug_format(false);
}
return underlying_.parse(ctx);

```

In `[format.tuple]`:

-1- For each of `pair` and `tuple`, the library provides the following formatter specialization where *pair-or-tuple* is the name of the template:

```

namespace std {
    template<class charT, formattable<charT>... Ts>
    struct formatter<pair-or-tuple<Ts...>, charT> {

        ...

        constexpr void set_brackets(basic_string_view<charT> opening,
                                    basic_string_view<charT> closing);

        constexpr formatter();

        template<class ParseContext>
        constexpr typename ParseContext::iterator
            parse(ParseContext& ctx);
    };
}

```

...

```

constexpr void set_brackets(basic_string_view<charT> opening, basic_string_

```

-6- *Effects*: Equivalent to:

```

opening-bracket_ = opening;
closing-bracket_ = closing;

```

```

constexpr formatter();

```

Effects: For each element *e* in *underlying_*, if *e.set_debug_format(true)* is a valid expression, calls *e.set_debug_format(true)*.

```
template<class ParseContext>
constexpr typename ParseContext::iterator
    parse(ParseContext& ctx);
```

-7- *Effects:* Parses the format specifier *s* as a *tuple-format-spec* and stores the parsed specifiers in **this*. The values of *opening-bracket_*, *closing-bracket_*, and *separator_* are modified if and only if required by the *tuple-type*, if present. ~~For each element *e* in *underlying_*, if *e.set_debug_format()* is a valid expression, calls *e.set_debug_format()*.~~

-8- *Returns:* An iterator past the end of the *tuple-format-spec*.

§ 6. Acknowledgements

Thanks to Barry Revzin and Mark de Wever for pointing out issues with debug formatting of ranges and tuples.

§ References

§ Informative References

[FMT]

Victor Zverovich; et al. [The fmt library](https://github.com/fmtlib/fmt). URL: <https://github.com/fmtlib/fmt>

[LIBCXX]

“libc++” C++ Standard Library. URL: <https://libcxx.llvm.org/>

[LWG3776]

Mark de Wever. [Avoid parsing format-spec if it is not present or empty](https://cplusplus.github.io/LWG/issue3776). URL: <https://cplusplus.github.io/LWG/issue3776>

[N4917]

Thomas Köppe; et al. [Working Draft, Standard for Programming Language C++](https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2022/n4917.pdf). URL: <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2022/n4917.pdf>

[P2286]

Barry Revzin. [Formatting Ranges](https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2022/p2286r8.html). URL: <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2022/p2286r8.html>